How-To: Void-Based Platformer Level Design Toolkit

A Modular Component Toolkit for 3D Platformer Level Design in Unreal Engine 5.

Roan Mulder - 240100

Table of Contents

Overview 2
Project Structure 2
Prerequisites & Project Setup 4

Learning the Elements
Element 1: Simple Movement (BPAC_SimpleMovement) 6
Element 2: Simple Rotation (BPAC_SimpleRotation) 7
Element 3: Disappearing Platform (BPAC_DisappearingPlatform) 9
Element 4: Bouncing Platform (BPAC_BouncingPlatform) 11
Element 5: Spline Moving Platform (BP_SplineMovingPlatform) 14
Element 6: Platforming Void Actor (BP_VoidActor) 19
Element 7: End Collectable (BP_EndCollectable) 20
Element 8: Regular Collectable (BP_RegularCollectable) 21

Utilizing the Elements
Complete Level Setup Guide 22
Adding New Content to the Toolkit 23
Troubleshooting 26
AI Acknowledgement 30

Overview

Welcome to the Void-Based Platforming Toolkit! In this document you will find every element this toolkit contains, with an in-depth explanation on setting them up, how they work and even some examples on how you could use each element.

This Toolkit takes elements taken from Super Mario Sunshine's Secret Courses and replicates their behavior in Unreal Engine 5. This toolkit is for the following users:

- **Beginners in Unreal Engine 5** that want to dabble in level design without having to do too much blueprinting.

- **Fans of 3D platformers** like Super Mario Sunshine or A Hat in Time that want to truly recreate the Secret Courses/Time Rift levels.

- **Advanced Unreal Engine 5 users** that want to expand upon an existing platformer toolkit.

Project Structure

Naming Conventions

Components

Actor Components: BPAC_[ComponentName]

- BPAC_SimpleMovement
- BPAC_SimpleRotation
- BPAC_DisappearingPlatform
- BPAC_LaunchPlatform

Blueprint Actors

Blueprint Actors: BP_[ActorName]

- BP_SplineMovingPlatform
- BP_VoidActor
- BP_EndCollectable
- BP_RegularCollectable

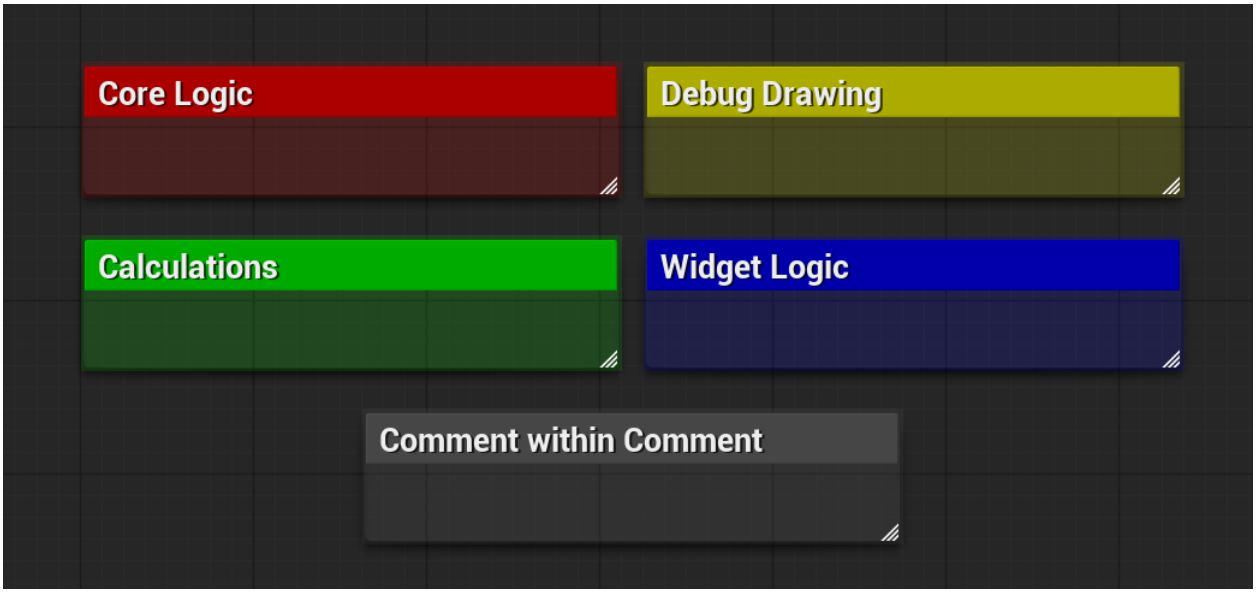
Widgets

Widget Blueprints: WB_[WidgetName]

- WB_FallOffScreen
- WB_LevelCompleteScreen
- WB_CollectableCounter

Comments & Color Coding

This toolkit uses a consistent color-coding system for Blueprint comments to help you quickly identify different types of logic:



- **Core Logic (Red)** - Essential event flows and main functionality
- **Debug Drawing (Yellow)** - Visual feedback and editor-only visualization
- **Calculations (Green)** - Mathematical operations and helper functions
- **Widget Logic (Blue)** - UI creation and management
- **Comment within Comment (Gray)** - Nested explanations or additional context

Prerequisites & Project Setup

Unreal Engine Version

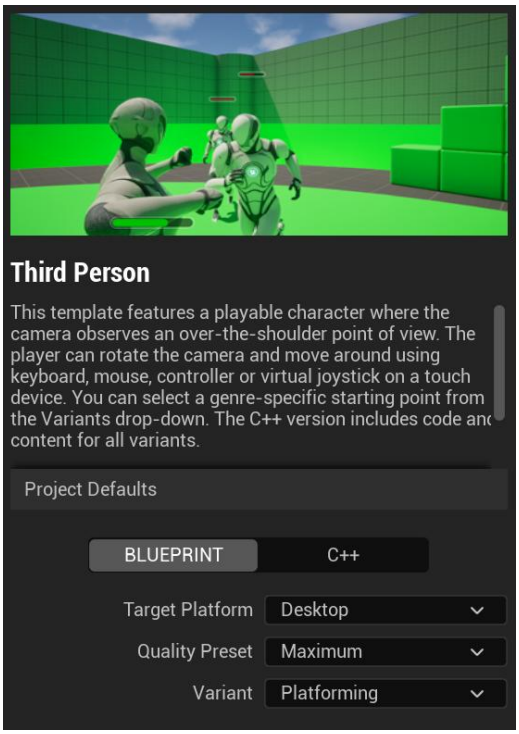
This toolkit is designed for **Unreal Engine 5.6 and above**. The project itself was created in **Unreal Engine 5.6.1**, but a newer version should also work with the project.

Project Setup

This toolkit was developed as a standalone project using Unreal Engine's Third Person Platforming Template. To use the toolkit in your own project, follow these setup steps.

Creating a New Project

1. Open Unreal Engine 5.6 (or newer version).
2. Select **Games** → **Third Person** template.
3. Check the **Platforming** Specification box.
4. Choose **Blueprint** as the project type.
5. Set your desired project location and name.
6. Click **Create**.



Importing the Toolkit

1. Locate the toolkit's **MyContent** folder in the provided project files.
2. Copy the MyContent folder into your new project's Content directory.
3. In Unreal Editor, the Content Browser will automatically detect the new files.
4. Navigate to **Content/MyContent/Blueprints** to access all toolkit components.

Verifying Installation

5. Create a new Empty Level or open an existing level.
6. Place a Static Mesh cube in the level and set it to **Movable**.
7. In the Details panel, click **Add** and type **BPAC**.
8. The toolkit is installed correctly if any of the Actor Components show up after typing BPAC.
9. For testing purposes, select **BPAC_SimpleRotation.**
10. Press Play to verify that the cube rotates.

Setting Up Your First Level

- In this How-To document you will find a **Complete Level Guide**, right after all the elements have been explained. Follow this guide to create your very first level. Context of all the elements is needed to properly create your first level. It is recommended to go through **all** the elements **before** creating your first level.

Project Organization

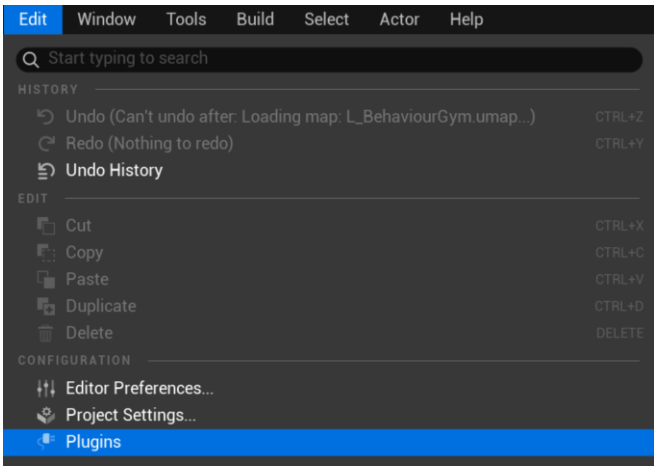
In the project, there are many folders that are related to the Third Person Character Platforming Variety Template by Unreal Engine itself. To find the actual content of the toolkit, go to **Content/MyContent.** Here you will find several folders:

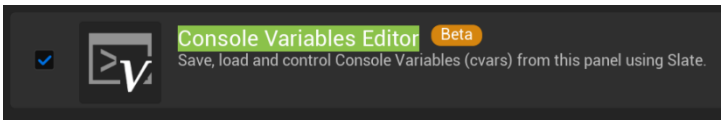
- Blueprints: this folder is divided into subfolders. CoreBehaviors (Contains the 5 core behaviors), EnumsAndStruct (contains all Enums that are used), LevelLogicActors (Actors that make the Level be a level) & Widgets (Menus and Counter).
- Maps: The Museum Level (L_BehaviorGym) and the Demo Level can be found here.
- Materials: Here you can find various Materials used by Components of the Toolkit.

Enabling Spline Visualization

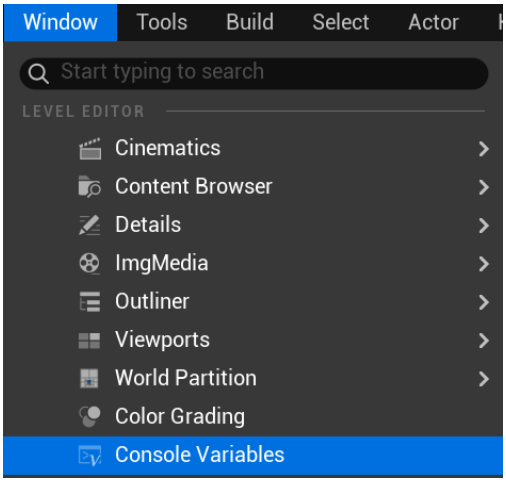
The Splines should be permanently visible in all states of the project from start-up.
If they are not:

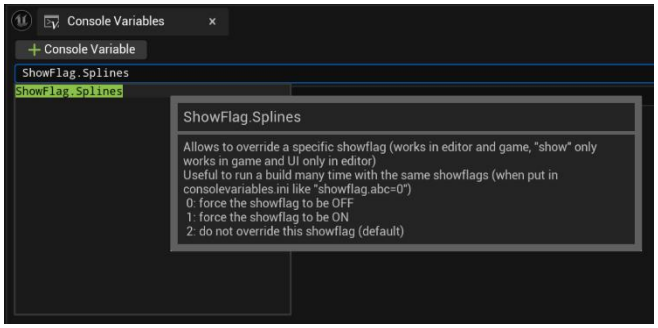
1. Go to Plugins and search for “Console Variables Editor”

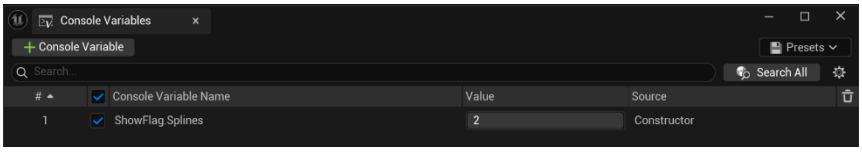




2. Open the Console Variables Menu, which is found in the Window dropdown menu.
3. Add a new console variable: **ShowFlag.Splines**
4. If this variable is not set to 2, set it to 2.







Element 1: Simple Movement (BPAC_SimpleMovement)

The **Simple Movement Actor Component** is a component that when applied to a **Movable** Static Mesh, will move said Static Mesh back and forth on a linear movement path from a Start Location (Wherever you place the Static Mesh) and an End Location (can be set with a variable). The movement can be aligned to either X, Y or Z axis. The Direction the movement heads towards first, can be set to the positive or negative direction of the aligned axis.

Adding the Component

7. Place a Static Mesh at a desired (starting) location.
8. Set Static Mesh to Movable.
9. In the details panel, click the Add button and type **BPAC_SimpleMovement.**
10. Configure Axis, Distance, and Starting Direction in the Details Panel.

Key Variables

- **Movement Axis (E_MovementAxis)**
  Aligns the movement to an Axis:
  Moves along X Axis - Left-Right Movement *(default)*
  Moves along Y Axis - Forward-Backwards Movement.
  Moves along Z Axis - Up-Down Movement.
- **Axis Direction (E_AxisDirection)**
  Heading to the Positive *(default)* or Negative of whatever Axis the Movement is set to.
- **Movement Distance (Float)**
  The Distance of Movement between Start & End Location *(default: 500.0).*
- **Movement Speed (Float)**
  The Speed of Movement between Start & End Location *(default: 100.0).*
- **Pause Duration (Float)**
  The amount of time the Mesh pauses at Start & End Location *(default 2.0).*

Design Benefits & Limitations

- Movement path is automatically set on placing the Static Mesh.
- User can easily duplicate the behavior and easily adjust the movement.
- **Can be combined with other Actor Components.** Combines best with Simple Rotation or Bouncing Platform Actor Components. **While a Disappearing Platform Actor Component works, is not recommended to combine with Simple Movement.**
- **Adjusting the Static Mesh's in-editor rotation will not adjust the Movement**, Movement Path will always be a straight line along the Movement's aligned axis.
- Lastly, a small quirk is that the Guide Arrow that visualizes the path of the simple movement, is only visible during Play-In-Editor.

Element 2: Simple Rotation (BPAC_SimpleRotation)

The **Simple Rotation Actor Component** is a component that when attached to a **Movable** Static Mesh, will rotate the Mesh. (Much like Unreal Engine's built in Rotating Movement component, only a lot more beginner friendly.)

The rotation can be aligned to either the X, Y or Z axis and its direction can also be changed to be either clockwise or counterclockwise, in context to the axis the rotation is aligned to.

Adding the Component

1. Place a Static Mesh at a desired location.
2. Set Static Mesh to Movable.
3. In the details panel, click the Add button and type **BPAC_SimpleRotation.**
4. Configure Axis, Rotation Direction, and Rotation Speed in the Details Panel.

Key Variables

- **Rotation Axis (E_RotationAxis)**
  Which axis to rotate around:
  Rotates along X Axis – Pitch Rotation. *(default)*
  Rotates along Y Axis - Roll Rotation.
  Rotates along Z Axis – Yaw/Horizontal Rotation.
- **Rotation Direction (E_RotationDirection)**
  Sets the direction of the Rotation to be either clockwise *(default)* or counterclockwise.
- **Rotation Speed (Float)**
  The speed of the Rotation *(default 30.0).*

Design Benefits & Limitations

- Rotation is completely relative to the Static Mesh, meaning the rotation works at any angle you set the static mesh to.
- Rotation direction easily swappable, no need to rotate the mesh manually to get the same angle with alternate rotation direction.
- Rotation is **only** directly aligned with the X, Y or Z axis. Diagonal Rotation relative to the world is not possible. No crazy rotations that are done with very specific angles and no double rotation, so no rotating in two different axes.
- Can be combined with other Components. However, it is **not recommended to combine the Simple Rotation component with the Bouncing Platform nor the Disappearing Platform components.**

Common Use Case Examples

Three examples of use cases for the Simple Rotation, commonly found in platformers.

Example 1: Simple Spinning Platform

Slower moving platform that spins horizontally that the player can easily stand on.

- **Rotation Axis:** Rotates along Z Axis.
- **Rotation Direction:** Clockwise or Counterclockwise (preference).
- **Rotation Direction:** 15.0.

Example 2: Tumbling Rectangle

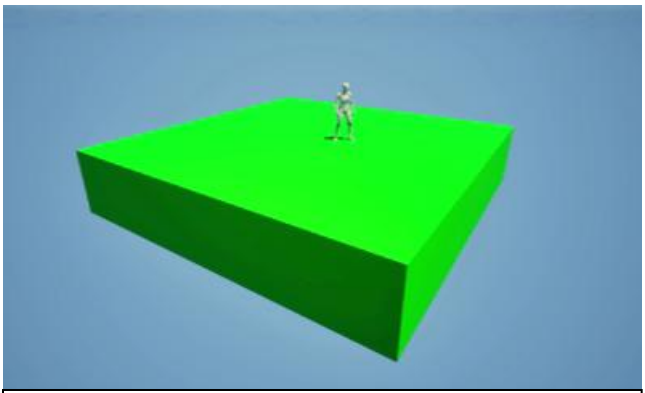
Taken from Super Mario Sunshine, a long rectangular platform that rotates along the short side, the player will have to time their jump as they get across the platform.

- **Rotation Axis:** Along X or Y Axis (depending on how the long side is created).
- **Rotation Direction:** Clockwise or Counterclockwise (preference).
- **Rotation Direction:** 15.0.

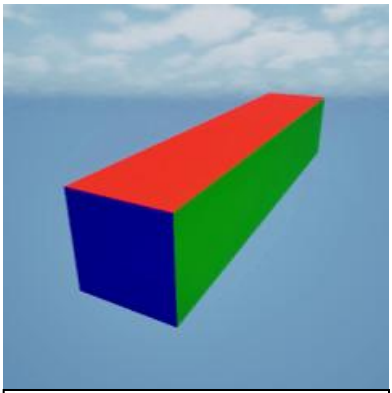
Example 3: Cogwork Platform

Taken from Super Mario Sunshine, big slow cogwheel shaped platform where its teeth are areas the player can easily land on that then get lifted upward.

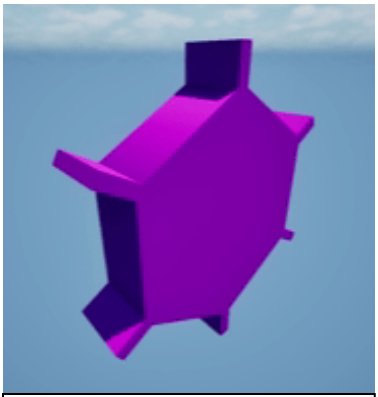
- **Rotation Axis:** Rotates along X Axis.
- **Rotation Direction:** Clockwise.
- **Rotation Direction:** 30.0.



Example 1



Example 2



Example 3

Element 3: Disappearing Platform (BPAC_DisappearingPlatform)

The **Disappearing Platform Actor Component** is a component that when attached to a Static Mesh, will have the Static Mesh act like a disappearing platform. The platform reappears after a set amount of time. The user can choose to have the platform disappear when the player steps on it and for it to reappear a set amount of time later, or have the platform disappear and reappear on a timer regardless of player interaction.

Adding the Component

1. Place a Static Mesh at a desired location.
2. Set Static Mesh to Movable.
3. In the details panel, click the Add button and type **BPAC_DisappearingPlatform.**
4. Configure Disappearance Start, Visible Duration, Disappearance Duration and Reappearance Duration in the Details Panel.

(For Alternating Platforms, follow steps 1 through 4 again, and for every second Static Mesh added to the level, set its Reverse Disappearance to True.)

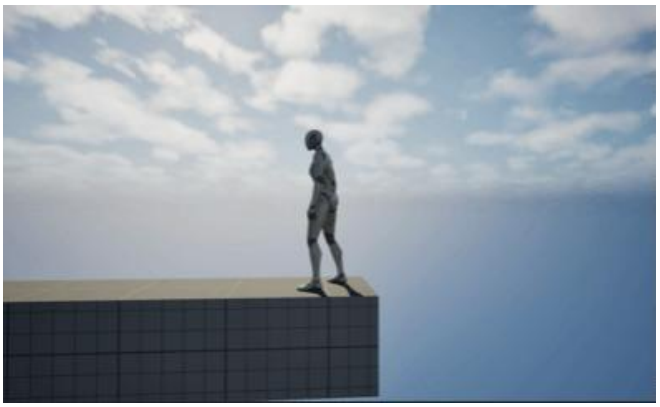
Key Variables

- **Disappearance Start (E_OnCollisionOrTimer)**
  How the platform starts disappearing:
  Disappears on Timer:
  Start disappearing on begin play, then disappears and reappears on a Timer, a set amount of time later.
  Disappears on Player Collision:
  Starts disappearing when player touches the Static Mesh, then reappears a set amount of time later.
- **Visible Duration (Float)**
  The amount of time where the platform remains visible before beginning to disappear (default: 2.0)
- **Disappearance Duration (Float)**
  The amount of time it takes for the platform to disappear completely (default: 2.0)
- **Reappearance Duration (Float)**
  The amount of time it takes for the platform to reappear instantly (default: 4.0).
- **Reverse Disappearance (Boolean)**
  Reverses the Disappearing Platform Logic. Used for Alternating Disappearing Platforms. Half of the platforms need this set to True, the other half to False. **Alternating Platforms only work when all float variables are of equal value.** (recommendation: Visible = 2.0, Disappearance = 2.0, Reappearance = 2.0).
- **(Advanced User Option) Disappearance Material (Material)**
  The Material used in the disappearing of the platform, advanced user can change this material with one of the same nature but different visual style, but this is **not recommended for beginner users**, beginner users just use the material provided (default: M_BlackFadeOverlay).

Common Use Case Examples

Three examples of use cases on the disappearing platform, common to platformers.

Example 1: Crumbling Bridge

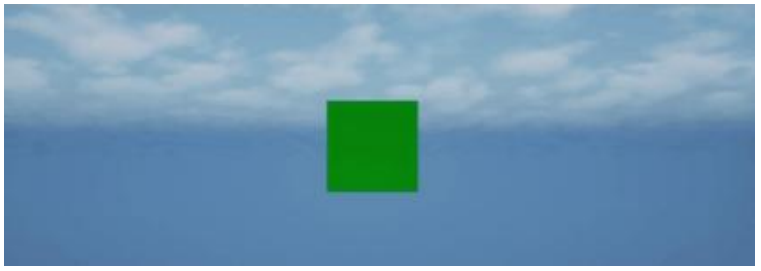


A series of disappearing platforms that instantly start disappearing as soon as the player makes contact, to give the feeling of a bridge crumbling from underneath the player, pushing the player to keep moving forward.

- **Visible Duration:** 0.1
- **Disappearance Duration:** 1.0.
- **Reappearance Duration:** 4.0.

Example 2: Alternating Disappearing Platforms

Series of disappearing platforms on a timer that are alternating in disappearance.
Only works if all variables are of equal value, across all platforms.



Disappearing Platforms A:

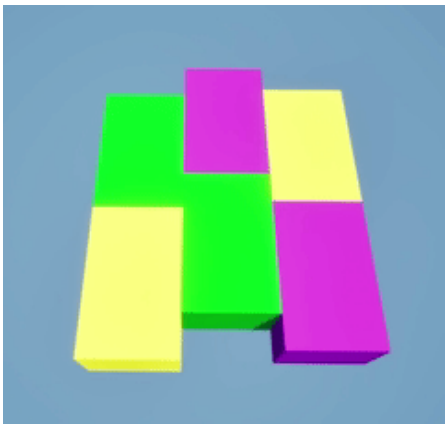
- **Visible Duration:** 2.0
- **Disappearance Duration:** 2.0.
- **Reappearance Duration:** 2.0
- **Reversed Disappearance:** False

Disappearing Platforms B:

- **Visible Duration:** 2.0
- **Disappearance Duration:** 2.0.
- **Reappearance Duration:** 2.0
- **Reversed Disappearance:** True

Example 3: Chaotic Disappearing-Reappearing Platforms

Taken from Super Mario Sunshine,
a group of platforms that all disappear and reappear in a chaotic pattern.
This is done by having each platform have a varying Visible Duration value but the same Disappearance Duration and Reappearance Duration value.



- **Visible Duration:** Various Values.
- **Disappearance Duration:** 2.0
- **Reappearance Duration:** 2.0

Element 4: Bouncing Platform (BPAC_BouncingPlatform)

The **Bouncing Platform Actor Component** is a component that turns a Static Mesh into one that bounces or launches the player. The component is split into two main modes, Simple Bounce and Balloon Bounce. *(Based on Balloons from A Hat in Time.)*

Simple Bounce launches the player straight into the air, Balloon Bounce launches the player in a forward arc. Balloon Bounce can be used by itself as a simple diagonal launch but can also be placed in such a way they chain together, to cross open spaces. The Balloon Bounce "pops" after the player touches it and reappears a set time later.

Adding the Component

1. Place a Static Mesh at a desired location.
2. Set Static Mesh to Movable.
3. In the details panel, click the Add button and type **BPAC_BouncingPlatform.**
4. Configure Bounce Type, Simple Bounce or Balloon Bounce.
    a. If Simple Bounce, configure Simple Bounce Variant Simple Bounce Height and Simple Bounce Cooldown (maybe Arrow Guide Offset).
    b. If Balloon Bounce, configure Balloon Bounce Direction, Balloon Bounce Distance, Balloon Bounce Height & Balloon Bounce Respawn Cooldown.

Key Variables

- **Bounce Type (E_LaunchType)**
  Determines the type of Bounce, either Simple Bounce or Balloon Bounce.

Simple Bounce Variables

- **Simple Bounce Variant (E_BounceType)**
  Determines the variant of the Simple Bounce: Single (default) or Continuous.

  Single Bounce launches the player on collision, but player needs to manually make contact again for the Static Mesh to launch the player again.
  (Unless Simple Bounce Cooldown is set to 0.5, which is instant rebounce)

  Continuous Bounce launches the player on collision and will automatically launch the player again, after the set Simple Bounce Cooldown has been completed.
- **Simple Bounce Height (Float)**
  Sets the height that the player is launched straight into the air (default 1000.0).
- **Simple Bounce Cooldown (Float)**
  The amount of time it takes for the Platform to fire the launch player node.
  If set to 0.5 it instantly launches the player no matter what variant (default 1.0).
- **Arrow Guide Offset (Float)**
  A debug guide arrow is drawn that roughly equates to launch path, relevant to the center of the Unreal Engine Male Mannequin Model. when changing the player character (model), set the arrow guide offset value to add or subtract to/from the arrow, to align with the new player character's center (default 0.0).

Balloon Bounce Variables

- **Balloon Bounce Direction (E_LaunchDirection)**
  Determines the direction that the Balloon Bounce launches the player.
  The direction the user can pick between are both the Positive and Negative directions of each axis.
  Positive X (default), Negative X, Positive Y, Negative Y, Positive Z, Negative Z.
- **Balloon Bounce Distance (Float)**
  The distance that the Balloon Bounce launches the player horizontally, can also be seen as the direction distance. (default 1000.0) A Debug Guide Arrow is drawn to help the player visualize the horizontal distance, showing the player where to place the next Balloon Platform if they want a chain with no height gain.
- **Balloon Bounce Height (Float)**
  The distance that the Balloon Bounce launches the player vertically, can also be seen as the height of the launch arc. (default 800.0) A Debug Guide Arrow is drawn to help visualize the height of the launch. This arrow is displayed at the end of the Arrow from Balloon Bounce Distance, showing the player where they must place the next Balloon Platform if they want an upward moving chain.
- **Balloon Bounce Respawn Cooldown (Float)**
  The amount of time it takes for the Balloon Platform to reappear after it has been “Popped” (default 1.0)

Design Benefits & Limitations

- Alternate player model consideration in the Simple Bounce. Since the Simple Bounce’s Guide Arrow is where the player will roughly end up, this arrow is adjustable in case a user makes changes or fully swap the player model. This has been thought of beforehand even though it’s quite a specific edge case.
- Flexibility in Horizontal and Vertical Movement for Balloon Bounce Platforms. The user can experiment easily with different Distance and Height values, so they could create a very horizontal chain or a very vertical chain.
- The guide arrows that visualize Distance and Height for Balloon Bounce, adjusts automatically with the value of the two variables, which makes experimentation a lot easier to do.
- One quirk the Balloon Bounce Platforms have, is that they activate the moment the player touches their Static Mesh:

  In the event a user wants to change direction within the chain, while this is possible, **the balloon in the new direction needs to be slightly adjusted**, as the launch fires from the **edge** of the balloon. So, **if the new direction balloon is aligned perfectly with the old direction balloon, the player will shoot right past it.**

- **Can be combined with other Actor Components but only combines well with the Simple Movement Component.**

Common Use Case Examples

Four examples of use cases of the Bouncing Platform, found in platformers.

Example 1: Trampoline

A simple Mesh that when the player lands on it, will bounce the player straight up. When the player lands again, they will instantly be launched back up again, like a trampoline or any other bouncy surface.

- **Simple Bounce Variant:** Single or Continuous (in this case it won't matter).
- **Simple Bounce Height:** 1000.0
- **Simple Bounce Cooldown:** 0.5 (instant bounce).
- **Guide Arrow Offset:** 0.0 (depends on scale of player model).

Example 2: Rhythm Based Bouncing Floor

A long flat Static Mesh (or multiple shorter/smaller ones) that will bounce the player on a timer (Bounce Cooldown). The player is bounced upwards every few seconds or so. When combined with hazards this can provide quite a tricky challenge.

- **Simple Bounce Variant:** Continuous
- **Simple Bounce Height:** 1000.0
- **Simple Bounce Cooldown:** 3.0
- **Guide Arrow Offset:** 0.0 (depends on scale of player model).

Example 3: Balloon Platform Bridge Chain (No Height Gain)

A series of Balloon Platforms placed at the same height. By having the balloons at the same height, the player will get launched far into a direction, without gaining a lot of height. **(Consecutive Balloons must be placed at the end of the horizontal arrow.)**

- **Balloon Bounce Direction:** Positive X (Direction can be any, up to the user).
- **Balloon Bounce Distance:** 1000.0
- **Balloon Bounce Height:** 800.0
- **Balloon Bounce Respawn Cooldown:** 1.0

Example 4: Balloon Platform Staircase Chain (Height Gain)

A series of Balloon Platforms, each placed higher than the last. By having the balloons increase in height each time, the player is provided an airborne staircase of sorts. Great for having the player cross a far distance both horizontally as vertically.
(Consecutive Balloons must be placed at the tip of the smaller vertical arrow.)

- **Balloon Bounce Direction:** Positive X (Direction can be any, up to the user).
- **Balloon Bounce Distance:** 1000.0
- **Balloon Bounce Height:** 800.0
- **Balloon Bounce Respawn Cooldown:** 1.0

Element 5: Spline Moving Platform (BP_SplineMovingPlatform)

The **Spline Moving Platform** is a complete actor that moves along a spline path with support for multiple movement modes, rotation behaviors, pause points, and player-triggered activation. This is the most complex component in the toolkit.

Setting up Spline Mesh & Spline Movement

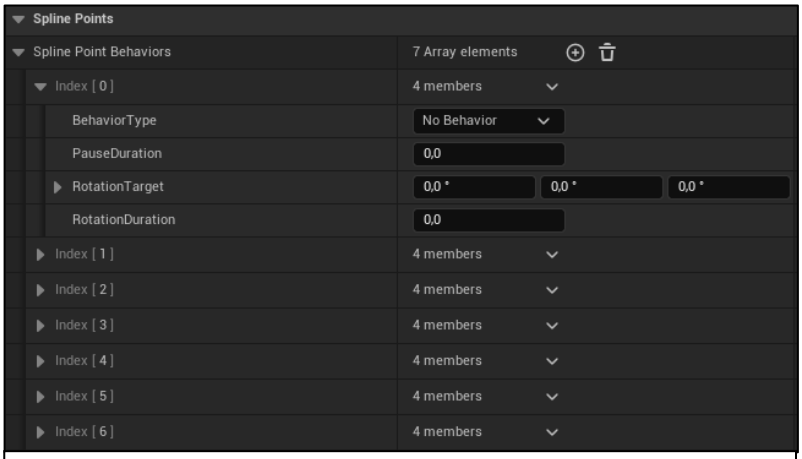
1. Drag **BP_SplineMovingPlatform** into your level
2. Drag a Static Mesh actor in the level and set it to Movable.
3. Select the Spline Actor, in the Spline Menu section of the details panel, in Spline Mesh, use the “Pick Actor from Scene” button and click on the Static Mesh actor you dragged into the level, to attach it to the Spline.
4. Configure Spline Movement Start, Movement Type, Movement Speed and Wait Duration.

Setting up Spline Points & Spline Point Behaviors

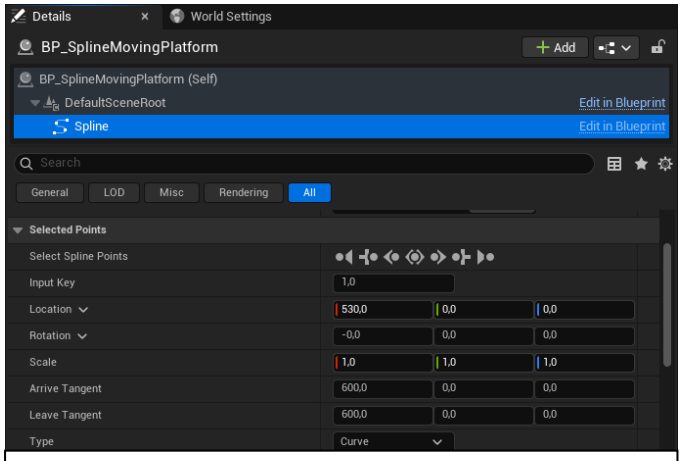
5. Add Spline Points by right click on the section of the Spline where you want Spline Points to go.
   (You can tweak the position with precision by selecting a spline point and then in the Spline Component part of the Actor, go to “Selected Points” section.)

6. In the Spline Points section of the details panel, click the Add Element + button to add an index to the array. Do this for each Spline Point placed onto the Spline. **(Index Zero = Start Point and Last Index = End Point)**

7. Per Spline point, configure the Behavior Type to No Behavior, Pause Behavior or Rotation Behavior. **(Do not alter anything in the Start Point and End Point!)**
   a. If set to Pause Behavior, configure the Pause Duration.
   b. If set to Rotation Behavior, configure the Rotation Target and Rotation Duration.



Example of a Spline with 5 Spline Points
And example of Spline Point Behavior Menu.
7 Spline Points total,
Not counting Start Point = Index [0] & End Point = Index [6].



Example of Spline Component’s Spline Point Selection Menu.
In example, user has clicked on Spline Point Index [1].

Key Variables

Spline Variables

- **Spline Mesh (Static Mesh)**
  This is the Static Mesh that gets attached to the Spline, so the actual object that will be moved along the Spline and influenced by the behavior within the Spline Points. This Spline Mesh can be any Static Mesh that is set to moveable. The Static Mesh will snap to the Start Point of the Spline no matter where in the level the Static Mesh is located.
- **Spline Movement Start (E_SplineStartMode)**
  Determines the way the Spline movement will start. Either on Begin Play (default) or on Player Collision. **If set to Begin Play**, the Pause Duration variable (further down the list) will pause the movement for a set amount of time before it truly begins moving. **On Player Collision**, the Spline will not start moving the Spline Mesh until player contact has been made, and when contact is made, it instantly begins moving.
- **Movement Type (E_SplineMovementType)**
  Determines how the Spline will move the Spline Mesh. There are three types of movement: One-Time Movement, Ping-Pong Movement (default) and Back To Start Movement.

  One-Time Movement will start the movement of the Spline Mesh and when the Spline Mesh reaches the end of the Spline, the Spline Mesh stays there permanently (until the level is restarted).

  Ping-Pong Movement acts very similar to One-Time Movement, but instead of the Spline Mesh stopping at the End Point, it instead reverses direction and begins heading back to the Start Point. And at the Start Point it goes back to the End Point, creating an endless Ping-Pong Movement.
  Pauses at both Start and End Point (using the Wait Duration variable).

  Back To Start Movement also acts like One-Time Movement, but instead of stopping, when the Spline Mesh reaches the end, it stops and begins to fade into invisibility, only to reappear at the Start Point of the spline after a set amount of time, where it will then start moving to the End Point again.
  Pauses at both Start and End Point (using the Wait Duration variable).
- **Movement Speed (Float)**
  Determines the speed at which the Spline Mesh moves along the Spline. (default 100.0)
  If the speed is set too high, it might break rotation behavior Spline Points.
- **Wait Duration (Float)**
  Determines how long the Spline Mesh will wait at **both** Start Point and End Point of the spline (default 2.0) So by default, it waits 2 seconds at the Start Point and then 2 seconds at the End Point. **(Hence why Start & End Point should not be altered or have any behavior whatsoever.)**

Spline Point Variables

- **Spline Point Behaviors (S_SplinePointBehavior)**
  Determines the logic of having each Spline Point have its unique behavior. Responsible for the Array that Spline Points can be added to. Each Spline Point has the same 4 variables (which are found below) but they only work per Spline Point.
- **Behavior Type (E_SplinePointBehavior)**
  Determines what type of behavior a Spline Point will contain. There are three options provided: No Behavior (default), Pause Behavior, Rotation Behavior.

  No Behavior does nothing to the Spline Mesh.
  A Spline Point with No Behavior draws a white sphere onto the Spline.
  (Start Point and End Point are drawn with a red sphere instead.)

  Pause Behavior makes it so that when the Spline Mesh crosses the Spline Point, the Spline Mesh pauses its movement for a set amount of time. (PauseDuration.)
  A Spline Point with Pause Behavior draws a green sphere onto the Spline.

  Rotation Behavior makes it so that when the Spline Mesh crosses the Spline, it begins rotating according to the rotation set in the Rotation Target Variable. The time it takes for the Spline Mesh to finish its rotation, depends on the Rotation Duration Variable.
  A Spline Point with Rotation Behavior draws a Cyan Sphere where the rotation starts, and a Cyan Cube where the rotation finishes, onto the Spline.
- **Pause Duration (Float)**
  Determines how long the Spline Mesh is paused when on a Spline Point with Pause Behavior. (default 2.0)
- **Rotation Target (Rotator)**
  Applies relative rotation to the Spline Mesh when it crosses a Spline Point with Rotation Behavior. The user has free reign on how the rotation is implemented. Rotation Target allows the user to rotate the Spline Mesh in multiple axes as once. (Example: X: 90.0 Y: 0.0 Z: -90.0) **Rotations are automatically reversed if Spline Mesh is moving back to Start Point.** Any rotation the user implements, will be mirrored when Spline Mesh is moving back.
- **Rotation Duration (Float)**
  The time it takes for the Spline Mesh to complete the Rotation set by the Rotation Target variable. (default 2.0)
- **Mirror Rotation on Reverse (Boolean)**
  Determines whether on Reverse Movement (from Ping-Pong Movement) the Rotation values are mirrored/reversed, where True = Mirrored (default False).

Design Benefits & Limitations

- Clear indication of which Spline Point does what, each behavior is color-coded.
- A limitation is that the Spline creation part of the actor is still the built-in Unreal Engine Spline Component, which has its own whole set of editing tools. These are not as well integrated with the Spline Point Behavior system. **Caution is advised with some of the settings in the Spline Component's details panel.**
- Actor Components can still be attached to the Spline Mesh. **Do not attach Simple Movement or Simple Rotation, this might break the Spline Movement and Spline Point Rotation.** Bouncing Platform is compatible.

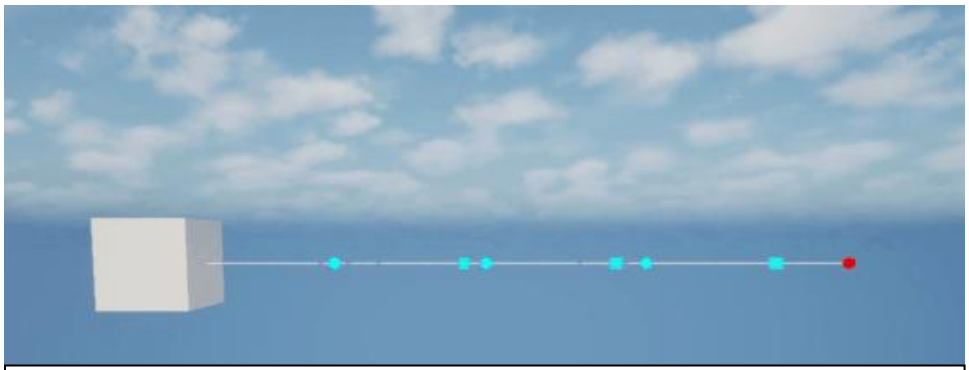
Common Use Case Examples

Three examples of how the Spline Moving Platform is used, common to platformers.

Example 1: Simple Rotation Cube Path (Horizontal)

A large cube that follows a straight path, but as the cube moves along the path it rotates in various directions in quick consecutive rotations. (From Super Mario Sunshine.)

- Spline drawn in a straight line aligned on either the X or Y axis.
- **Spline Movement Start:** Start on Player Collision.
- **Movement Type:** Ping-Pong Movement.
- **Movement Speed:** 100.0
- **Wait Duration:** 4.0
- **Spline Point Behaviors:** Rotation Behavior
    - **Spline Point 0:**
        - Start Point, No Behavior.
    - **Spline Point 1:**
        - **Rotation Target:** X: -90.0, Y: 0.0, Z: 0.0
        - **Rotation Duration:** 3.0.
    - **Spline Point 2:**
        - **Rotation Target:** X: 0.0, Y: 90.0, Z: 0.0
        - **Rotation Duration:** 3.0.
    - **Spline Point 3:**
        - **Rotation Target:** X: 0.0, Y: 0.0, Z: -90.0
        - **Rotation Duration:** 3.0.
    - **Spline Point 4:**
        - End Point, No Behavior.

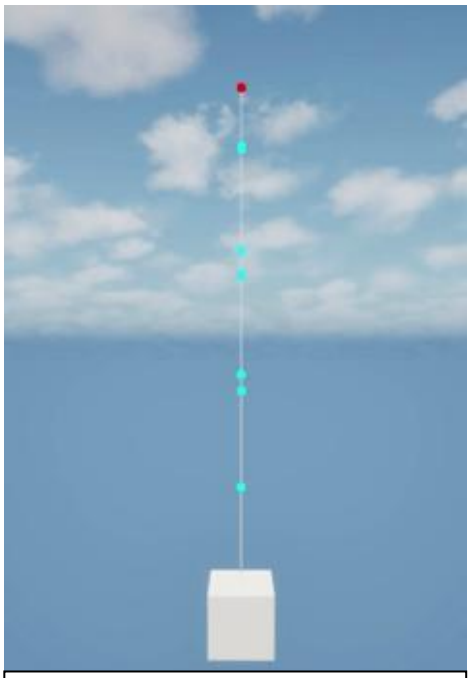


Visualization of Example 2, showcasing all the directions the mesh gets rotated and where the Spline Points roughly would be placed. (Video is sped up to fit GIF file size.)

Example 2: Complex Rotation Cube Path (Rising)

A large cube that follows a straight path upward, but as the cube moves along the path it rotates in various directions in quick consecutive rotations, some being more complex consisting of diagonal rotation in two axes at once. (From Super Mario Sunshine.)

- Spline drawn in a straight line aligned on the Z Axis.
- **Spline Movement Start:** Start on Player Collision.
- **Movement Type:** Ping-Pong Movement.
- **Movement Speed:** 100.0
- **Wait Duration:** 4.0
- **Spline Point Behaviors:** Rotation Behavior
    - **Spline Point 0:**
        - Start Point, No Behavior.
    - **Spline Point 1:**
        - **Rotation Target:** X: -90.0, Y: 0.0, Z: 0.0
        - **Rotation Duration:** 3.0.
    - **Spline Point 2:**
        - **Rotation Target:** X: 90.0, Y: 90.0, Z: 0.0
        - **Rotation Duration:** 3.0.
    - **Spline Point 3:**
        - **Rotation Target:** X: 0.0, Y: -90.0, Z: -90.0
        - **Rotation Duration:** 3.0.
    - **Spline Point 4:**
        - End Point, No Behavior

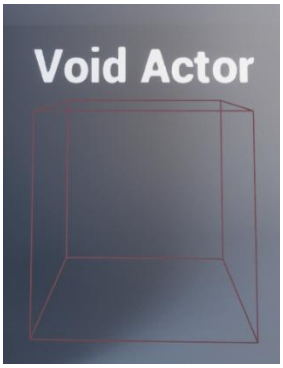


Visualization of Example 3: Showcasing upwards movement & complex rotations (Video is sped up to fit GIF file size.)

Example 3: Pause-and-Go Obstacle Course

A platform on a linear path where the Spline Mesh stops at several points throughout the Spline, can be used for either enemy encounters the player must dodge, or an obstacle that's hard to avoid when the platform is paused.

- Spline drawn in a straight line aligned on either the X or Y axis.
- **Spline Movement Start:** Start on Player Collision.
- **Movement Type:** Back To Start Movement.
- **Movement Speed:** 200.0.
- **Wait Duration:** 2.0.
- **Spline Point Behaviors:** Pause Behavior
    - **Spline Point 0:**
        - Start Point, No Behavior.
    - **Spline Point 1:**
        - **Pause Duration:** 6.0.
    - **Spline Point 2:**
        - **Pause Duration:** 2.0
    - **Spline Point 3:**
        - **Pause Duration:** 6.0.
    - **Spline Point 4:**
        - End Point, No Behavior.



Element 6: Platforming Void Actor (BP_VoidActor)

The Platforming **Void Actor** handles player death when falling off platforms. When the player character overlaps with the Void Actor, a screen is shown, like in Super Mario Sunshine, before restarting the level after a set time.

The Void Actor works in tandem with the Platforming End Collectable.

Setup Instructions

1. Drag **BP_VoidActor** into the level.
2. Place the actor below the platforms of your level, at a distance you find satisfactory. (Suggestion: 1000 units below the platforms.)
3. Get the X and Y scale and increase these until the Void Actor is fully underneath all the platforms within the level.

How It Works

4. Player falls off a platform into the void and overlaps with the actor.
5. On overlap, player input is disabled.
6. The Fall Off Screen Widget (WB_FallOffScreen) is then displayed right after.
7. After a set amount of time, the current level restarts. (RestartDelay Float)

Key Variable

- **Restart Delay (Float)**
  The Restart Delay float determines the amount of time it takes before the Fall Off Screen is shown and the level to restart.

Customization

The fall off screen widget provided is very simplistic in nature. The project is set-up for this widget to easily be replaced with a custom one:

- Create your own widget blueprint for the fall off screen.
- In the code of BP_VoidActor, there are clear commented sections where you can replace the Fall Off Screen widget.

(If you’re not well versed with widgets, you can also choose to change the text in the existing Fall Off Screen widget blueprint. By default, it should say: “you fell!”.



Element 7: End Collectable (BP_EndCollectable)

The Platforming **End Collectable** actor makes it so that the level is completed. The level is completed as soon as the player overlaps with the End Collectable, “collecting it” if you will.
The End Collectable works in tandem with the Void Actor.

Setup Instructions

1. Drag **BP_EndCollectable** into your level.
2. Position it at the end of your platforming challenge.
3. Drag the Void Actor in the level and set it up following its own instructions.

How It Works

4. On Begin Play, the Void Actor is found in the level by the End Collectable.
5. When player overlaps with the End Collectable, it disables the Void Actor.
6. On overlap, End Collectable is set invisible and player input is disabled.
7. Level Complete Screen Widget (WB_LevelCompleteScreen) gets displayed.
8. After a set amount of time, the next level is loaded. (NextLevelDelay)

Key Variables

- **Next Level Delay (Float)**
  The Next Level Delay float determines the amount of time it takes before the Level Complete Screen is shown and the next level is loaded.
- **Requires Collection (Boolean)**
  For functionality with Regular Collectable (See Element 8), if set to true, requires the collection of all regular collectables to have the end collectable appear.
- **Collectable Name (String)**
  For functionality with Regular Collectable (See Element 8). Determines what the name of the Regular Collectable is named in the Regular Collectable Counter.

Design Details

- Void Actor gets disabled through a simple disabling of its collision on collection of the end collectable.
- For this to properly work, only one BP_VoidActor should be present within a level.
- The End Collectable’s Material (M_PlatformEndCollectable) spins around using a Material Function (MF_RotateActor) to save a tiny bit of performance.

Customization

- The Level Complete Screen can be replaced by your own Level Complete Screen Widget.
- The Mesh of the End Collectable can be replaced by any mesh of your preference, although if it’s a big Mesh, make sure to adjust the Sphere Collision of the actor.

Element 8: Regular Collectable (BP_RegularCollectable)



The regular collectable is the younger brother to the End Collectable. Acting like a generic collectable when placed into the level. However, when checking the “Requires Collection” Boolean in the End Collectable, these generic collectables become red and will all have to be collected for the End Collectable to appear in the level.

The Regular Collectable works in tandem with the End Collectable if the user chooses to use the Regular Collectable’s Functionality.

Setup Instructions

1. Drag **BP_RegularCollectable** into your level.
2. Position multiple Regular Collectables throughout your level.
3. Drag BP_EndCollectable into your level and place it at the end of your level.
4. If you want End Collectable to be dependent on collecting regular collectables, check the “Requires Collection” box in End Collectable’s details panel. Also set the name of the Regular Collectable that will be displayed.

How it works

5. When setup properly, the Regular Collectables should have changed material on Begin Play. (Red by default.) A Counter Widget (WB_CollectableCounter) should also be on screen on Begin Play.
6. The total number of Collectables set by the counter, is equal to the number of Regular Collectables placed within the level.
7. Every Regular Collectable collected adds to the Counter.
8. Once all Regular Collectables have been collected, the counter disappears and the End Collectable appears where it has been placed in the level.

Key Variable

- **Require Collection Material (Material)**
  The Material that the Regular Collectable has when Requires Collection is set to true within the End Collectable (default: M_CollectableRed)

Design Details

- The Regular Collectable is designed in such a way, that the user has a lot of freedom in what they want to do with it. It can be a simple guiding path for the player, or a challenge for the player. The latter if the collection becomes required.

Customization

- The Mesh and Material of the Regular Collectable can be swapped with any mesh of the user’s choice. This includes the Required Collection Material, which is editable in the details panel for ease of use.
  Although the logic is **in BP_EndCollectable**, the Counter for the Regular Collectable that shows up, can be replaced with a custom counter widget, much like the other widgets. This info fits better in this section than in the previous.

Complete Level Setup Guide

Here's how all the components work together to create a complete void-based platforming level:

Level Creation Workflow

Step 1: Create Void Environment

1. Create a new level (Basic or Empty)
2. Delete default ground plane.
3. Add simple skybox or solid color background.
5. Place **BP_VoidActor** and scale it so it covers the full extent of your level.

Step 2: Build Platform Layout

4. Drag a simple Static Mesh in for your starting platform.
5. Drag a Player Start into the level and place it on your starting platform.
6. Add challenge platforms using components:
   BPAC_SimpleMovement for simple moving platform challenges.
   BPAC_SimpleRotation for various kinds of simple rotating/spinning platforms.
   BPAC_DisappearingPlatform for timing challenges or pressure to keep moving.
   BPAC_LaunchPlatform for simple trampolines or more complex balloon chains.
   BP_SplineMovingPlatform for complex paths with rotating and/or pausing.

Step 3: Add Goal

6. Once the level is built to satisfaction add a **BP_EndCollectable** to the level to top the level off.
7. Make sure to create a new level that goes into the logic of the End Collectable. (If you don’t, its set to restart the current level by default.)

Step 4: Add Collectables

8. Add as many **BP_RegularCollectable** actors to the level as you wish, wherever you deem their location best.
9. You can choose to toggle “Requires Collection” in **BP_EndCollectable** to turn the Regular Collectables into Collectables that are required to collect to have the End Collectable appear.

Adding New Content to the Toolkit

The toolkit is designed to be modular and easy to use for Beginning users of Unreal Engine. Advanced users can choose to expand the system in several ways.

Replacing UI Widgets

The toolkit includes placeholder widgets for the Fall Off screen and Level Complete Screen. These can easily be replaced with custom designs.

1. Create your own custom Widget Blueprint to replace the placeholder.
2. Open BP_VoidActor for the Fall Off Screen Widget and open BP_EndCollectable for the Level Complete Screen Widget.
3. In the Event Graph for both actors, Find the Create Widget node, which should be clearly labelled and therefor easy to find.
4. Change the Class dropdown to your own custom widget.
5. Compile, save and est to verify your widget displays correctly.

Adding New Spline Point Behaviors

The Spline Moving Platform currently supports Pause and Rotation behaviors. Advanced users can add additional behavior types by extending the Enum and Struct.

Step 1: Extend the Enum

1. Open the **E_SplinePointBehavior** Enum.
2. Add a new value for your behavior.

Example: Speed Boost (Spline Point temporarily increases in movement speed).

Step 2: Add Variables to Struct

3. Open the **S_SplinePointBehavior** struct.
4. Add variables needed for your behavior.

Example: For Speed Boost, add the **SpeedMultiplier** variable (Float).

Step 3: Implement Behavior Logic

5. Open **BP_SplineMovingPlatform**.
6. Find the **Switch on E_SplinePointBehavior** node in the point-crossing detection logic.
7. A new execution pin will have appeared for your new Enum value.
8. Connect this new execution pin to the behavior logic you want to apply.

Example: Speed Boost: multiply the **Current Movement Speed** by **SpeedMultiplier**.

Possible Behavior Examples

Some examples of possible behaviors that could be added to expand the toolkit:

- **Speed Boost:** Temporarily increase movement speed when crossing the point.
- **Slow Zone:** Temporarily decrease movement speed when crossing the point.
- **Scale Change:** Increases or decreases the size of a mesh over time.
- **Sound Trigger:** Play an audio cue when crossing the point (combinable).
- **Early Reversal:** Force the platform to change direction mid-path.

Creating Derivative Components

Advanced users can create specialized versions of existing components by using Blueprint inheritance.

1. Right-click an existing component (example: **BPAC_DisappearingPlatform**).
2. Select Create Child Blueprint Class.
3. Name following the convention: **BPAC_[YourVariantName]**.
4. Override functions or add new logic in the child blueprint.

Example derivative ideas:

- **BPAC_ColorCyclingPlatform:**
  Changes color to warn before disappearing.
- **BPAC_ConditionalPlatform:**
  Only appears when specific game conditions are met.

Adding Visual and Audio Polish

The toolkit focuses on core functionality. Visual and audio effects can be added to enhance the experience.

Recommended Addition Points

Sound Effects:

- **Disappearing Platform:**
  Add audio cue when the platform has fully disappeared. (Example: Bubble Pop)
- **Bouncing Platform:**
  Add audio cue that plays whenever a player is launched. (Example: Boing sound)
- **Spline platform rotation:**
  Something to notify that a platform is rotating.
  (Example: Mechanical grinding or rumble.)
- **Platforming Void Actor:**
  Add audio cue for when the player overlaps with the Void Actor.
  (Example: Falling whistle.)
- **End Collectable:**
  Something to notify the End Collectable has been collected.
  (Example: Success jingle or sparkle sound.)

Particle Effects:

- **Disappearing Platform:** Poof or dissolve particle
- **Bouncing Platform:** Impact burst on bounce
- **End Collectable:** Shimmer or sparkle effect

Material Effects:

- Glowing edges on active platforms.
- Warning flash before a platform disappears.

Expanding on Level Management

The current system restarts the level on death and transitions to a new level on completion (which is done without the use of a Game Instance Blueprint).

The Death system is made simple on purpose, to make it easier for more complex logic to be added later.

Lives System

1. Create a Game Instance Blueprint.
2. Add a Lives Remaining integer variable.
3. In the Void Actor, decrement Lives instead of immediately restarting the level.
4. Only restart when Lives reach zero.

Checkpoint System

5. Create a **BP_Checkpoint** actor that stores the player spawn location.
6. Place the actor in the locations you want checkpoints to go.
7. In the Void Actor's Event Graph, before the level restarts, add logic that moves the Player Start to the most recent checkpoint.
8. In the Game Instance, add logic that returns the Player Start to the original position before restarting from zero lives.
9. Add visual feedback when checkpoints are activated.

Level Progression

10. Create an Array of Level Names in the Game Instance Blueprint.
11. Track the Current Level Index.
12. In the End Collectable's Event Graph, replace the Open Level (by Name) node with logic that loads the next level in the array.

Important Notes for Extension

- **Maintain modularity:**
  Keep components independent when possible.
- **Expose variables:**
  Make new parameters Instance Editable so they appear in the Details panel.
- **Add tooltips:**
  Document what each new variable does and document the variable's constraints.
- **Follow naming conventions:**
  Use BPAC_, BP_, E_, S_, WB_ prefixes as appropriate.
- **Use color-coded comments:**
  Maintain the Red/Yellow/Green/Blue/Gray system for consistency.
- **Test combinations:**
  When adding components to the same actor, make sure to test thoroughly, so that they don't conflict and/or break logic.

Troubleshooting

General Platform Issues

Problem: Player gets pushed off moving platforms

- This is behavior that was found, but since this toolkit is for void-based platforming, it wasn't an issue worth fixing.
- Just make sure that when building a level, you don't have a platform push the player character while they're standing on something.

Simple Movement Issues

Problem: Platform doesn't move at all

- Verify the Static Mesh is set to Movable (not Static or Stationary).
- Check that Movement Speed is greater than 0.
- Ensure Distance is set to a value greater than 0.

Problem: Movement seems jittery or stutters

- This can occur if the platform is colliding with other objects in the level.
- Ensure there's clear space for the full movement path.
- Check that no other movement components or physics are affecting the mesh.

Simple Rotation Issues

Problem: Rotation direction is opposite of what I expected

- Try switching the Rotation Direction from Clockwise to CounterClockwise (or vice versa).
- The visual rotation depends on which axis you're viewing from - what looks clockwise from one angle appears counterclockwise from another.

Problem: Mesh rotates strangely or not aligned with chosen axis

- The rotation is relative to the mesh's local orientation, not world space.
- Verify the mesh is oriented correctly in the level.
- Check that the Static Mesh is set to Movable.

Disappearing Platform Issues

Problem: Platform doesn't fade smoothly / fades incorrectly

- Ensure your material is set to Translucent blend mode.
- Verify the material has a parameter named "FadeAlpha" that controls opacity.
- Check that the FadeAlpha parameter is properly connected to Opacity in the material.

Problem: Platform disappears but never reappears

- Verify Trigger Mode is not set to "Disappear On Player Collision" with Reappearance Duration of 0.
- If using Timer mode, check that Reappearance Duration is greater than 0.

Problem: Alternating platforms sync up instead of alternating

- All platforms must have identical timing values
  (Visible Duration, Disappearance Duration, Reappearance Duration).
- Verify that exactly half of the platforms have bReverseDisappearance set to True.
- Recommended values for clean alternating: 2.0 for all three durations.

Problem: OnPlayerCollision mode doesn't trigger

- Verify the Static Mesh has collision enabled.
- Check that the player character's collision channels are set up correctly.
- Ensure Trigger Mode is set to "Disappear On Player Collision".

Bouncing Platform Issues

Problem: Player doesn't launch when touching platform

- Ensure the actor has a Box Collision component.
- Verify Launch Velocity is greater than 0.
- Check that Simple Bounce Mode is set correctly (Single Bounce or Continuous Bounce).
- Make sure the Static Mesh is set to Movable.

Problem: Balloon bounces only work once / can't chain bounces

- Verify Bounce Type is set to Balloon Bounce (not Simple Bounce).
- Check that Max Bounces is greater than 1.
- Ensure the player is holding the jump button on subsequent bounces.
- Verify Velocity Multiplier Per Bounce is not set to 0.

Problem: Launch direction goes the wrong way

- Check the Launch Direction enum value matches your intended direction.
- The direction is relative to world space, not the platform's orientation.
- Try switching between Positive and Negative variants to reverse direction.

Problem: Balloon platforms disappear and never reappear

- This is intended behavior for Balloon mode.
- The platform reappears after the Respawn Duration has elapsed.
- If it's not reappearing, check that Respawn Duration is greater than 0.

Spline Platform Issues

Problem: Platform doesn't move along the spline

- Verify Spline Movement Start is set correctly (Begin Play or Player Collision).
- If set to Player Collision, ensure Use Collision Trigger is enabled and the player has touched the platform.
- Check that Movement Speed is greater than 0.
- Ensure the spline has at least 2 points.
- Verify the Spline Mesh is set to a valid Static Mesh.

Problem: Rotation behaviors don't trigger at spline points

- Verify the Point Index matches an actual spline point (remember that Index 0 = first point).
- Check that Behavior Type is set to Rotation Behavior.
- Ensure Rotation Duration is greater than 0.
- If movement speed is very high, the platform might overshoot the detection threshold - try lowering Movement Speed.

Problem: Pause behaviors don't work at spline points

- Verify the Point Index matches an actual spline point.
- Check that Behavior Type is set to Pause Behavior.
- Ensure Pause Duration is greater than 0.

Problem: Spline visualization not showing in editor

- Open the Console Variables menu (Window → Console Variables).
- Check that ShowFlag.Splines is set to 2.
- Alternatively, enable Show Debug Spheres on the BP_SplineMovingPlatform actor.

Problem: Platform moves but rotation produces unexpected diagonal motion

- This can occur with sequential multi-axis rotations.
- For predictable results, use single-axis rotations (90, 0, 0) or (0, 90, 0).
- Compound rotations (45, 45, 0) will produce diagonal tumbling by design.
- Ensure the Shortest Path option is enabled in the rotation lerp logic (this is set by default).

Problem: Collision trigger doesn't activate the platform

- Verify Use Collision Trigger is set to True.
- Ensure Spline Movement Start is set to Player Collision.
- Check that the Spline Mesh has collision enabled and is set to Movable.
- The player must physically touch the mesh for the trigger to activate.

Platforming Void Issues

Problem: Death screen doesn't appear when falling

- Verify the widget class is set correctly in the BP_PlatformingVoid blueprint.
- Check that the void's box collision is large enough to catch falling players.
- Ensure the player character is set up to trigger overlaps with the void.

Problem: Level doesn't restart after falling into void

- Check that the Open Level node in BP_PlatformingVoid is properly connected.
- Verify the delay before restart is working (default 2 seconds).
- Ensure the current level name is being retrieved correctly.

Problem: Void triggers even after collecting the end goal

- This indicates the DisableVoid function isn't being called properly.
- Verify that BP_PlatformingVoid exists in the level.
- Check that BP_PlatformingEndCollectable is finding the void actor on Begin Play.
- Only one BP_PlatformingVoid should exist per level.

End Collectable Issues

Problem: "Accessed None" error for void reference

- Caused by not having the BP_VoidActor in your level.
- End Collectable and Void Actor work together in tandem.
- Just adding the BP_VoidActor should do the trick in fixing the problem.

General Issues

Problem: Multiple components on the same actor behave strangely

- Some component combinations may conflict (especially timing-based ones).
- Test combinations thoroughly before using in final levels.
- Simple Rotation + Simple Movement works well together.
- Disappearing Platform + Bouncing Platform may have timing conflicts.

Problem: Can't edit component values on multiple selected actors

- Unreal Engine 5 doesn't support multi-selecting actors to batch-edit component values.
- Instead: Configure one platform with desired settings, then duplicate (Alt+Drag or Ctrl+W).
- All duplicates will maintain the same component settings.

Problem: Guide spheres/arrows visible during gameplay

- Debug visualization is enabled by default for easier level design.
- To hide in final builds: Set Show Debug Spheres to False on spline platforms.
- Guide arrows for Simple Movement only appear during Play-In-Editor, not in packaged builds.

AI Acknowledgement

- This documentation was developed with assistance from Claude AI (Anthropic).
- An initial draft template was provided by Claude AI, which was then substantially rewritten by the author in their own words and style.
- The majority of troubleshooting entries were written by Claude AI and verified for accuracy by the author.
- All component implementations, examples, and technical content reflect the author's original work.